



IF3270 Pembelajaran Mesin

Attention & Transformer

Tim Pengajar IF3270



ATTENTION!

Central idea behind Attention is **not to throw away those intermediate encoder states** but to utilize all the states in order to **construct the context vectors** required by the decoder to generate the output sequence

Bahdanau, D., Cho, K. H., & Bengio, Y. (2015, January). Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations, ICLR 2015*.

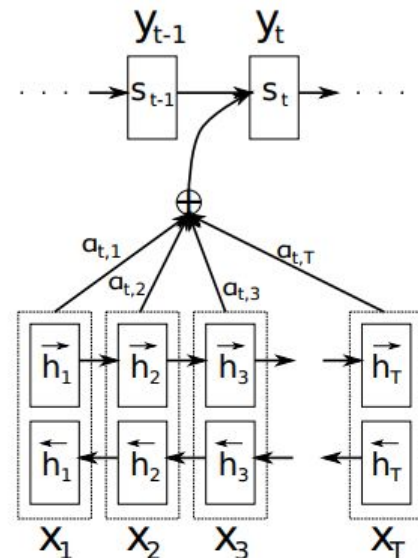
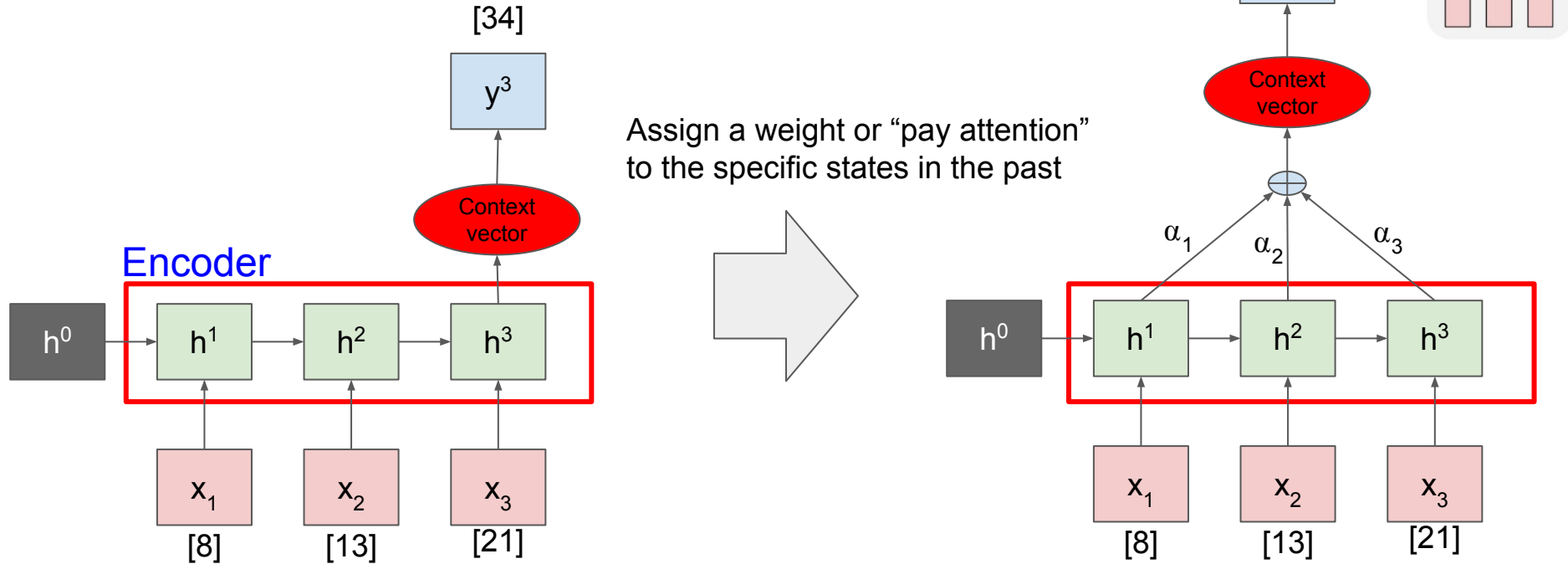


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Many to One with Attention



Many to One with Attention

Input: $\mathbf{x} = (x_1, \dots, x_{N_x})$

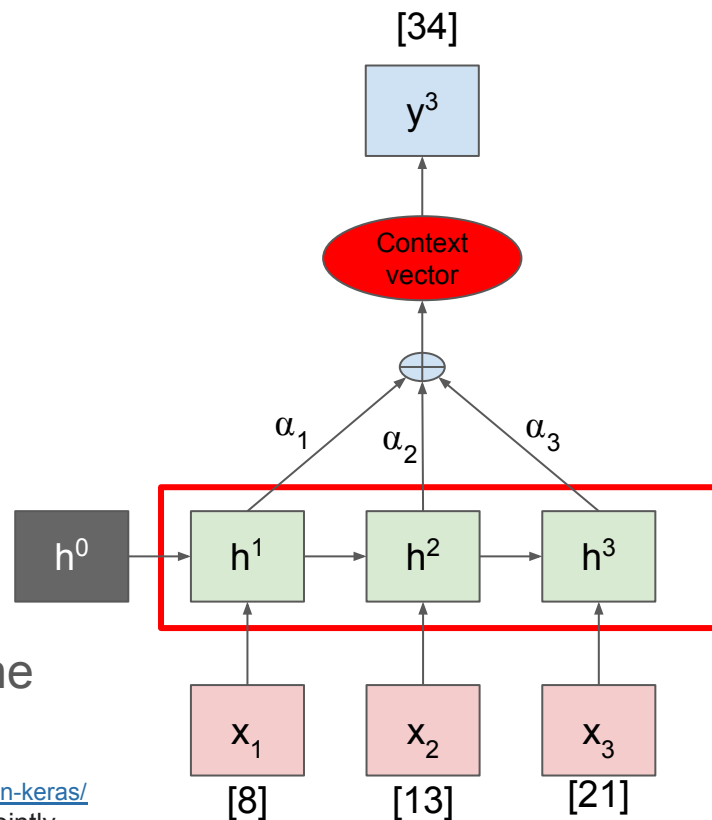
Hidden state at time j : $h_j = f(x_j, h_{j-1})$

Context vector c

$$c = \sum_{j=1}^{N_x} \alpha_j h_j$$

For Fibonacci series, the mean square error on the test set is lower with the attention layer.

<https://machinelearningmastery.com/adding-a-custom-attention-layer-to-recurrent-neural-network-in-keras/>
Bahdanau, D., Cho, K. H., & Bengio, Y. (2015, January). Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations, ICLR 2015*.



Many to One with Attention Architecture Example

Without attention

Model: "sequential_49"

Layer (type)	Output Shape	Param #
simple_rnn_52 (SimpleRNN)	(None, 2)	8
dense_62 (Dense)	(None, 1)	3

Total params: 11 (44.00 Byte)
 Trainable params: 11 (44.00 Byte)
 Non-trainable params: 0 (0.00 Byte)

Hidden_units = 2

Dense_units = 1

Time_steps = 20

#Params = (1+2+1)*2+(2+1)*1=11

Implementation of certain library, for example: Keras

$$c = \sum_{j=1}^{N_x} \alpha_j h_j \Rightarrow c = \sum_{j=1}^{N_x} \alpha_j h_j + b_j$$

With attention

Model: "sequential_63"

Layer (type)	Output Shape	Param #
simple_rnn_66 (SimpleRNN)	(None, 20, 2)	8
attention_11 (attention)	(None, 2)	22
dense_76 (Dense)	(None, 1)	3

Total params: 33 (132.00 Byte)
 Trainable params: 33 (132.00 Byte)
 Non-trainable params: 0 (0.00 Byte)

Hidden_units = 2

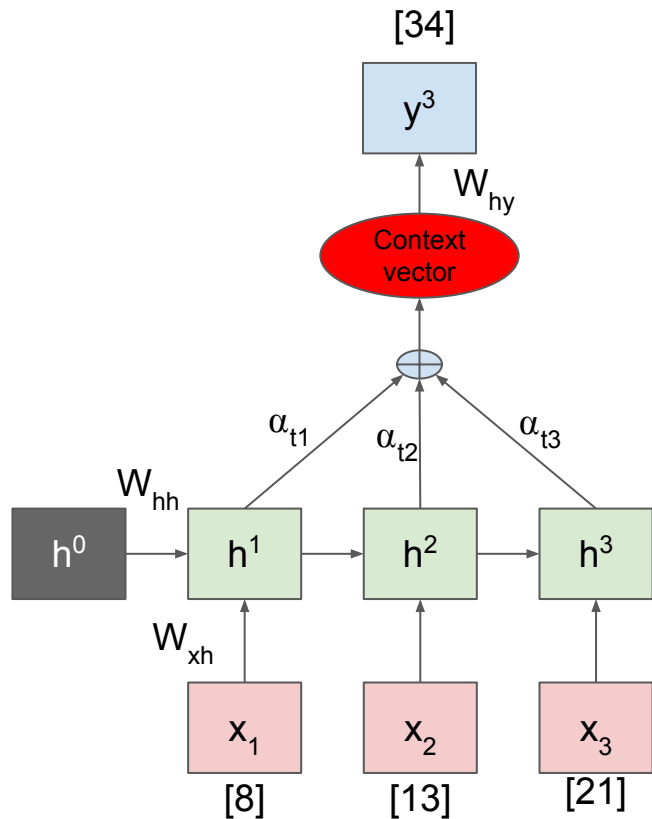
Dense_units = 1

Time_steps = 20

Attention_units=1

- Attention weight: 20 $\alpha_{t1} \dots \alpha_{t_{ts}}$
Shape: (time_steps, attention_units)
- Attention bias: 2
Shape: (hidden_units, attention_units)

Many to One with Attention: Forward Propagation



$$y^{(t)} = f_y(W_{hy}c + b_{hy})$$

$$c = \sum_{j=1}^{N_x} \alpha_j h_j$$

$$h^{(j)} = f_h(W_{xh}x^{(j)} + W_{hh}h^{(j-1)} + b_{xh})$$

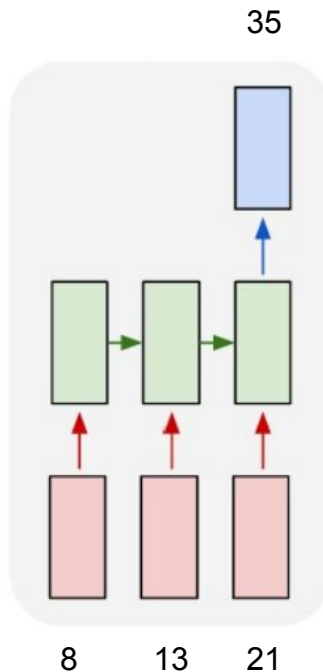
RNN with Attention: Context Vector

timestep	alpha_t	ht_neuron1	ht_neuron2
1	-0.6759	0.009	0.404
2	-0.6923	0.063	0.087
3	-0.7468	0.083	0.270
4	-0.7456	0.124	0.102
5	-0.8025	0.146	0.186
...	...	...	...
19	0.7771	0.341	-0.002
20	0.7768	0.344	-0.005

$$c = \sum_{j=1}^{N_x} \alpha_j h_j$$

$$c = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^{N_x} \alpha_j h_{1j} \\ \sum_{j=1}^{N_x} \alpha_j h_{2j} \end{bmatrix}$$

Many to One: with vs without 1 Attention Unit



Without attention

$$h^{(1)} = \text{ReLU}(W_{xh}x^{(1)} + W_{hh}h^{(0)} + b_{xh})$$

$$h^{(2)} = \text{ReLU}(W_{xh}x^{(2)} + W_{hh}h^{(1)} + b_{xh})$$

$$h^{(3)} = \text{ReLU}(W_{xh}x^{(3)} + W_{hh}h^{(2)} + b_{xh})$$

$$y^{(3)} = \text{Linear}(W_{hy}h^{(3)} + b_{hy})$$

Output: $y^{(3)}$

$$\# \text{Params} = (n_{\text{input}} + n_{\text{hidden}} + 1) * n_{\text{hidden}} + (n_{\text{hidden}} + 1) * n_{\text{output}}$$

t

With attention

$$h^{(1)} = \text{ReLU}(W_{xh}x^{(1)} + W_{hh}h^{(0)} + b_{xh})$$

$$h^{(2)} = \text{ReLU}(W_{xh}x^{(2)} + W_{hh}h^{(1)} + b_{xh})$$

$$h^{(3)} = \text{ReLU}(W_{xh}x^{(3)} + W_{hh}h^{(2)} + b_{xh})$$

$$c = \sum_{j=1}^{N_x} \alpha_j h_j$$

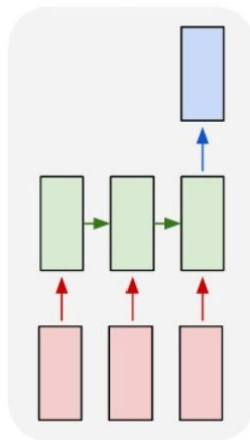
$$y^{(3)} = \text{Linear}(W_{hy}c + b_{hy})$$

Output: $y^{(3)}$

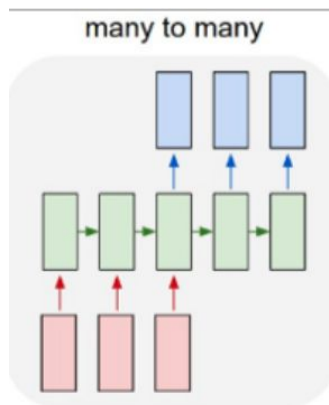
$$\# \text{Params} =$$

$$(n_{\text{input}} + n_{\text{hidden}} + 1) * n_{\text{hidden}} + n_{\text{timestep}} * n_{\text{hidden}} + (n_{\text{hidden}} + 1) * n_{\text{output}}$$

Attention: Many to One vs Many to Many



$$c = \sum_{j=1}^{N_x} \alpha_j h_j$$



$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

Context vector per timestep decoder

#Params =
 $n_{\text{timestep_enc}} + n_{\text{hidden}}$

#Params =
 $(n_{\text{timestep_enc}} + n_{\text{hidden}}) n_{\text{timestep_dec}}$

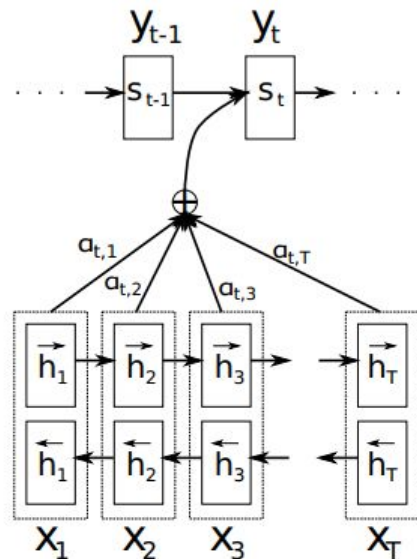


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Encoder Decoder without vs with Attention

Encoder:

$$h_j = f(x_j, h_{j-1})$$

$$h_j = f(x_j, h_{j-1})$$

Context Vector:

$$c = q(\{h_1, \dots, h_{N_x}\})$$

$$q(\{h_1, \dots, h_{N_x}\}) = h_{N_x}$$

$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{N_x} \exp(e_{tk})}$$

$$e_{tj} = \text{align}(s_{t-1}, h_j) = v_a^T \tanh(W_a s_{t-1} + U_a h_j)$$

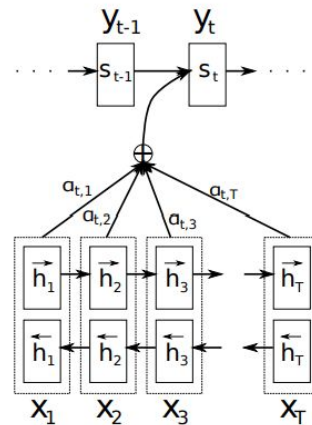
the process of “paying attention” can be learned during training

Decoder:

$$p(y_t | y_1, \dots, y_{t-1}, x) = g(y_{t-1}, s_t, c)$$

$$p(y_t | y_1, \dots, y_{t-1}, x) = g(y_{t-1}, s_t, c_t)$$

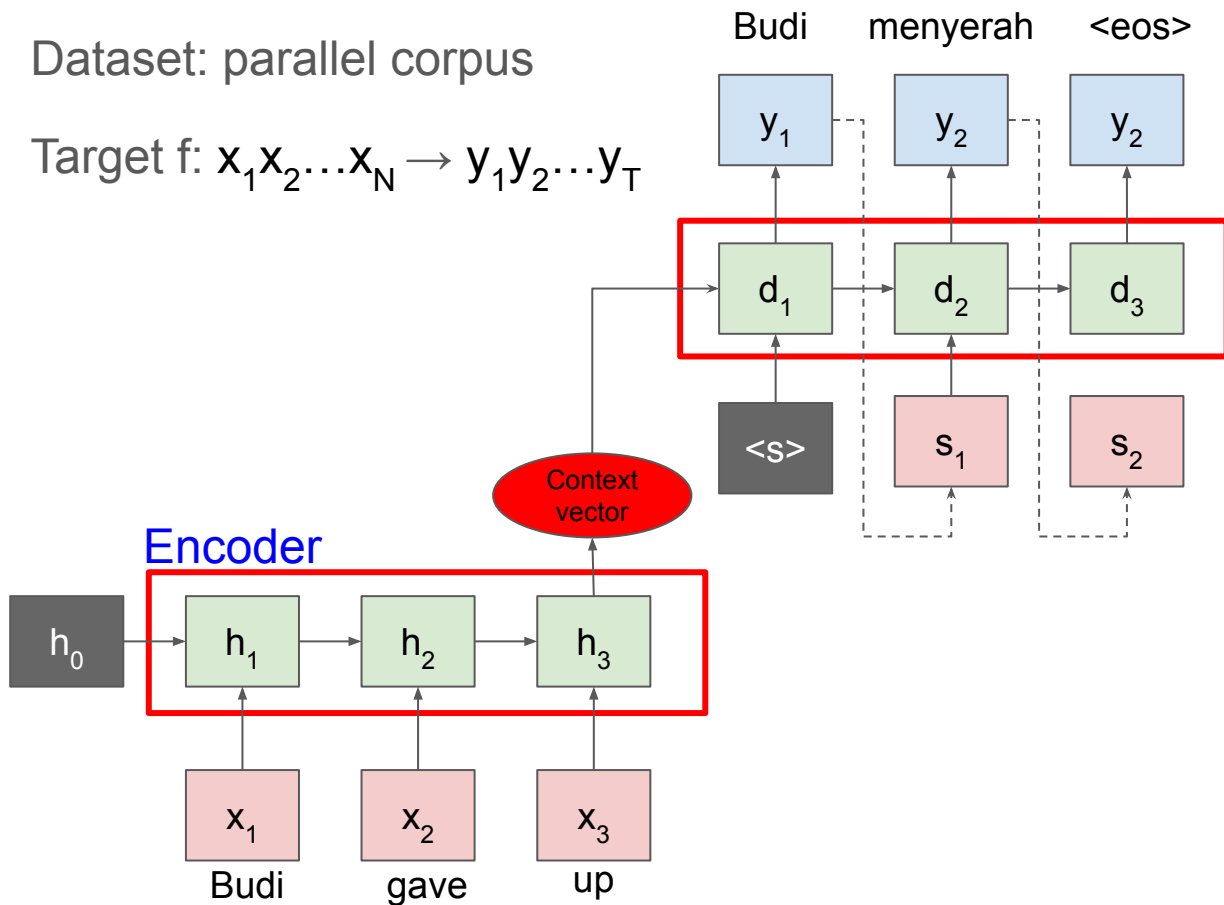
$$s_t = f(s_{t-1}, y_{t-1}, c_t)$$



Encoder - Decoder with Attention

Dataset: parallel corpus

Target f: $x_1 x_2 \dots x_N \rightarrow y_1 y_2 \dots y_T$



Decoder is trained to predict the **next word** y_t given the **context vector** c and all the previously predicted words $\{y_1, \dots, y_{t-1}\}$.

$$p(y_t | y_1, \dots, y_{t-1}, x) = g(y_{t-1}, s_t, c_t)$$

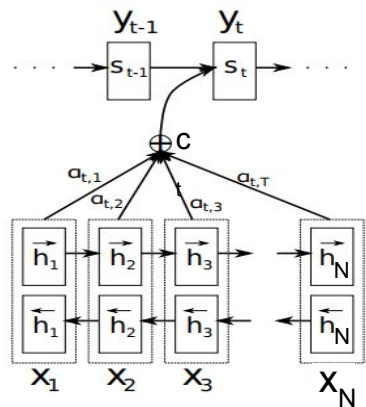
$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{N_x} \exp(e_{tk})}$$

$$e_{tj} = \text{align}(s_{t-1}, h_j)$$

$$= v_a^T \tanh(W_a s_{t-1} + U_a h_j)$$

Bahdanau vs Luong Attention Mechanism



$$S_{t-1} \rightarrow \alpha_t \rightarrow C_t \rightarrow S_t$$

$$p(y_t | y_1, \dots, y_{t-1}, x) = g(y_{t-1}, s_t, c_t)$$

$$s_t = f(s_{t-1}, y_{t-1}, c_t)$$

$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{N_x} \exp(e_{tk})}$$

$$e_{tj} = \text{align}(s_{t-1}, h_j) = v_a^T \tanh(W_a s_{t-1} + U_a h_j)$$

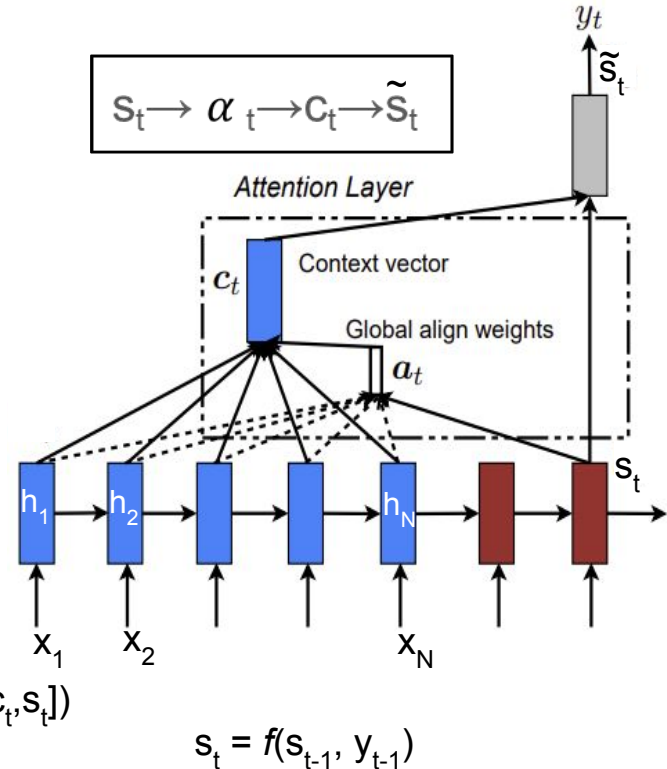
$$\begin{aligned} p(y_t | y_1, \dots, y_{t-1}, x) \\ &= g(\tilde{s}_t) \\ &= \text{softmax}(W_s \cdot \tilde{s}_t) \end{aligned}$$

$$\tilde{s}_t = f(c_t, s_t) = \tanh(W_c [c_t, s_t])$$

$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

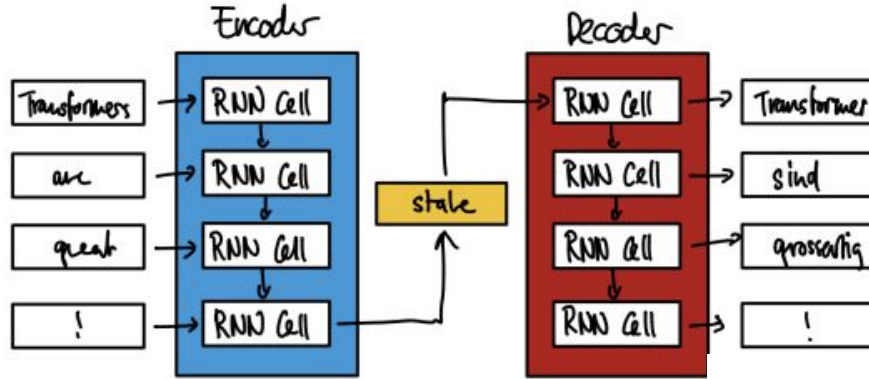
$$\alpha_{tj} = \frac{\exp(\text{score}(s_t, h_j))}{\sum_{k=1}^{N_x} \exp(\text{score}(s_t, h_k))}$$

$$\text{score}(s_t, h_j) = \begin{cases} s_t^T h_j & \text{dot} \\ s_t^T W_a h_j & \text{general} \\ v_a^T \tanh(W_a [s_t; h_j]) & \text{concat} \end{cases}$$

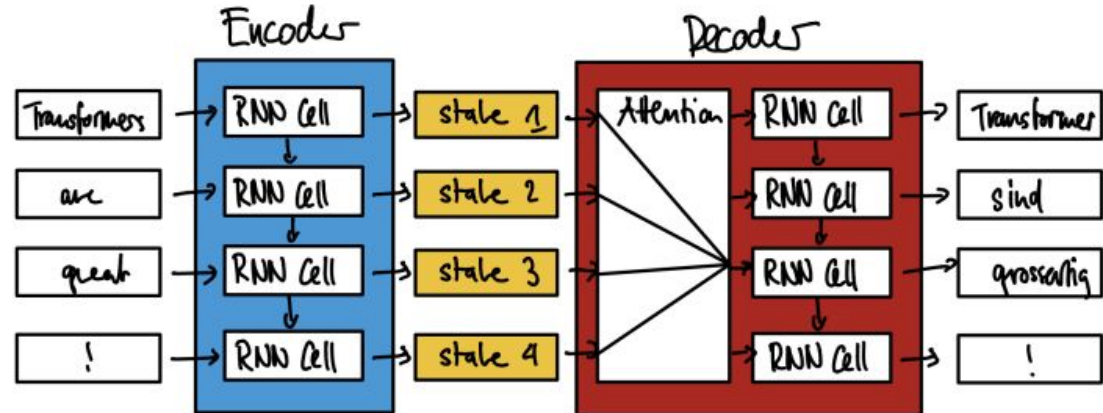


$$s_t = f(s_{t-1}, y_{t-1})$$

Vanilla RNN Encoder - Decoder without vs with Attention



What is the difference between these architectures ?



Tunstall, L., Von Werra, L., & Wolf, T. (2022). *Natural language processing with transformers*. " O'Reilly Media, Inc."

Scientific Breakthrough

Attention for Encoder-Decoder (Seq2Seq)

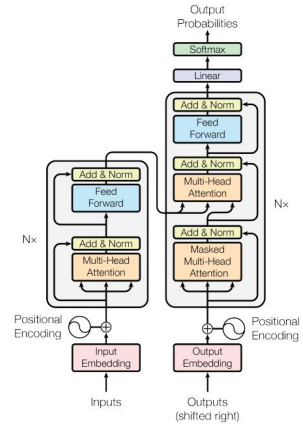
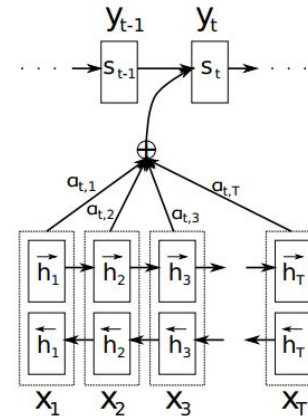
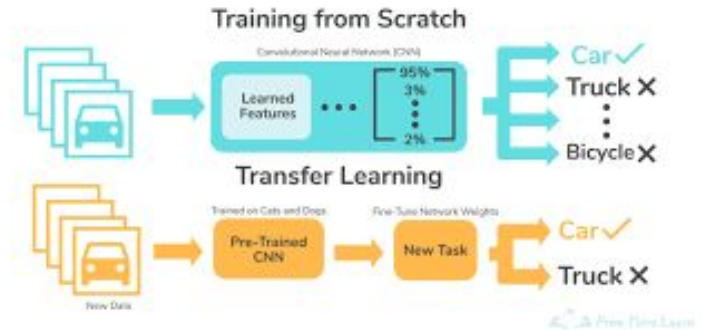
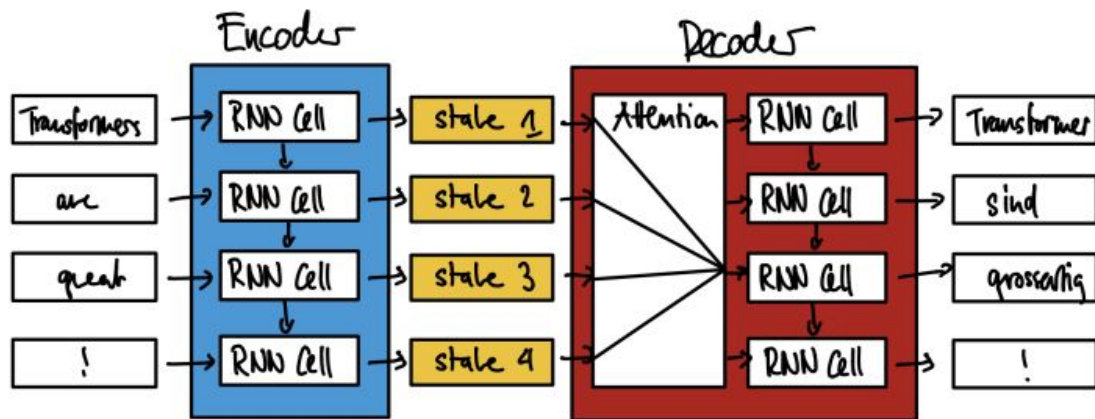


Figure 1: The Transformer - model architecture.

Transfer Learning

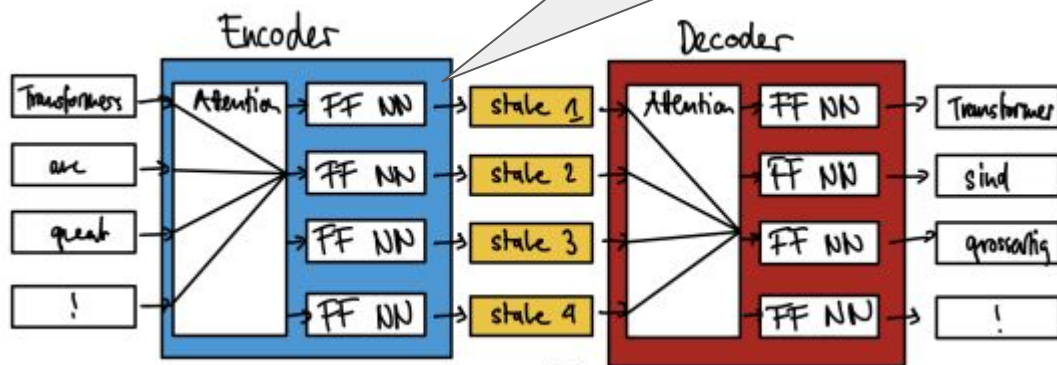


Vanilla RNN Enc-Dec with Attention vs Transformers



The Transformer architecture replaced the recurrent units inside the encoder and decoder entirely with **self-attention** layers and simple **feed-forward** networks.

all the tokens are fed **sequentially** (Vanilla RNN Enc-Dec) vs **in parallel** through the model (transformers)



Tunstall, L., Von Werra, L., & Wolf, T. (2022). *Natural language processing with transformers*. "O'Reilly Media, Inc."

Vanilla RNN Enc-Dec vs Transformers

- Moving from a **sequential** processing to a **fully parallel** processing **unlocked** strong computational **efficiency** gains allowing to train on orders of magnitude **larger corpora** for the same computational cost.
- At the same time, removing the sequential processing bottleneck of information makes the transformer architecture **more efficient** on several task that requires aggregating information over **long time spans**.
- The scaling laws of deep learning models: *larger models trained on more data in many cases yield better results*.
 - scaling models comes at the price of requiring large amounts of training data
- Transformer revolution started: **transfer learning**

Transformer: Dot Product Self Attention (1)

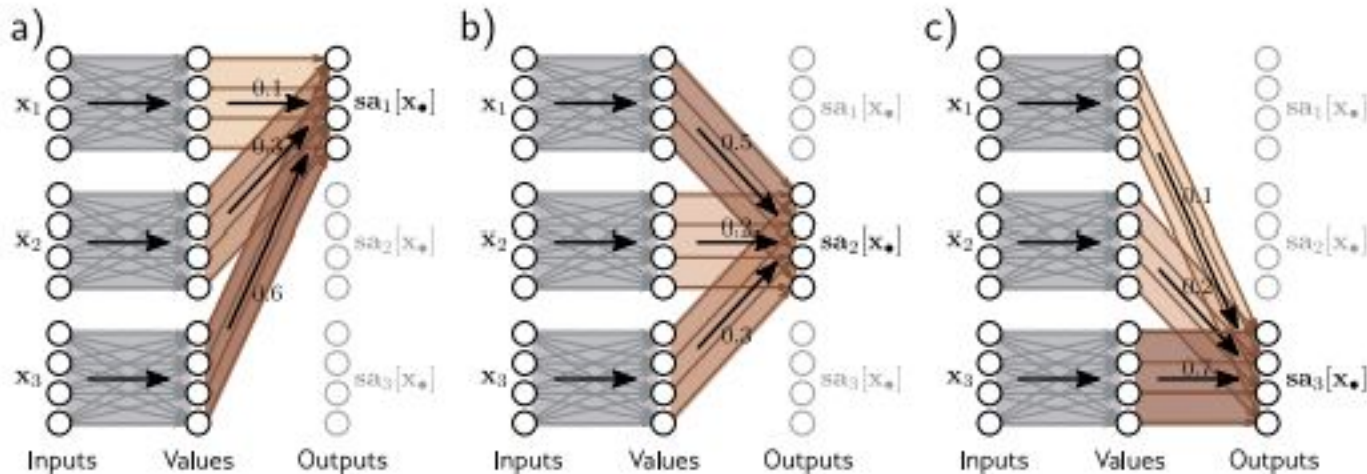
Adopted into Transformer: **Dot Product Self Attention**:

Previous: Luong Attention (dg dot product)

$$\alpha_t = \sum_{j=1}^N \alpha_{tj} h_j$$

$$\alpha_{tj} = \frac{\exp(\text{score}(s_t, h_j))}{\sum_{k=1}^{N_x} \exp(\text{score}(s_t, h_k))}$$

$$\text{score}(s_t, h_j) = s_t^\top h_j$$



x: input
v: value

α : bobot attention
sa: self attention

$$sa_n[x_1, \dots, x_N] = \sum_{m=1}^N a[x_m, x_n] v_m$$

$$v_m = \beta_v + \Omega_v x_m$$

$$\begin{aligned} a[x_1, x_1] &= 0.1 \\ a[x_2, x_1] &= 0.3 \\ a[x_3, x_1] &= 0.6 \\ sa_1[x] &= 0.1v_1 + 0.3v_2 + 0.6v_3 \end{aligned}$$

$$\begin{aligned} a[x_1, x_2] &= 0.5 \\ a[x_2, x_2] &= 0.2 \\ a[x_3, x_2] &= 0.3 \\ sa_2[x] &= 0.5v_1 + 0.2v_2 + 0.3v_3 \end{aligned}$$

$$\begin{aligned} a[x_1, x_3] &= 0.1 \\ a[x_2, x_3] &= 0.2 \\ a[x_3, x_3] &= 0.7 \\ sa_3[x] &= 0.1v_1 + 0.2v_2 + 0.7v_3 \end{aligned}$$

Transformer: Dot Product Self Attention (2) Self attention (dr satu input yg sama)

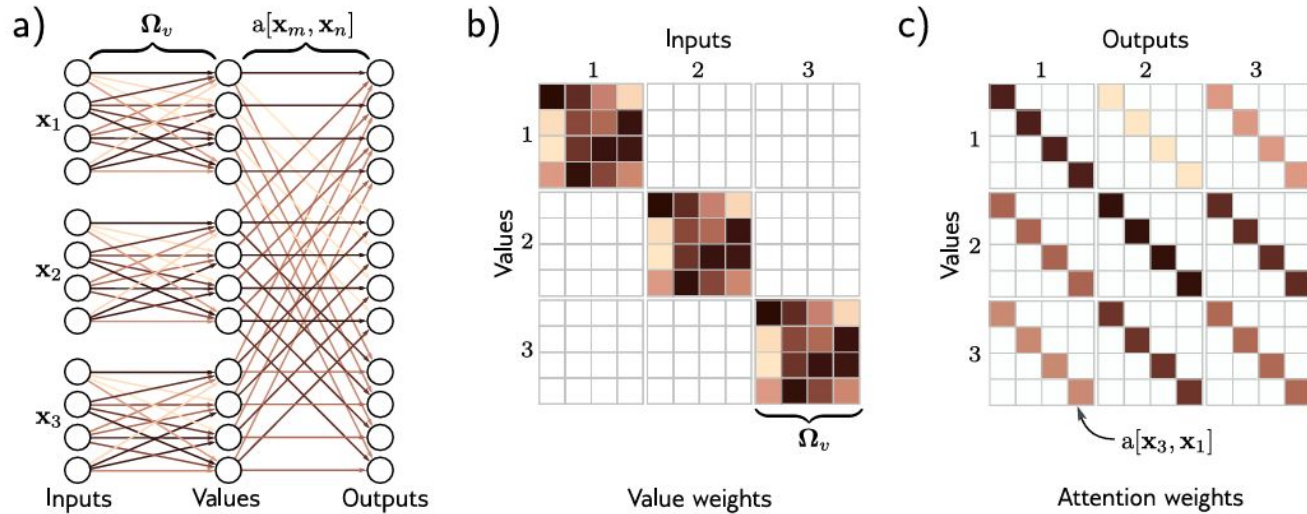


Figure 12.2 Self-attention for $N=3$ inputs $\mathbf{x}_n$, each with dimension $D=4$. a) Each input $\mathbf{x}_m$ is operated on independently by the same weights Ω_v (same color equals same weight) and biases β_v (not shown) to form the values $\beta_v + \Omega_v \mathbf{x}_m$. Each output is a linear combination of the values, with the attention weight $a[\mathbf{x}_m, \mathbf{x}_n]$ defining the contribution of the m^{th} value to the n^{th} output. b) Matrix showing block sparsity of linear transformation Ω_v between inputs and values. c) Matrix showing sparsity of attention weights relating values and outputs.

$$\mathbf{v}_m = \beta_v + \Omega_v \mathbf{x}_m$$

$$\text{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N] = \sum_{m=1} a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m$$

Attention diambil dari transformasi linear thd input:

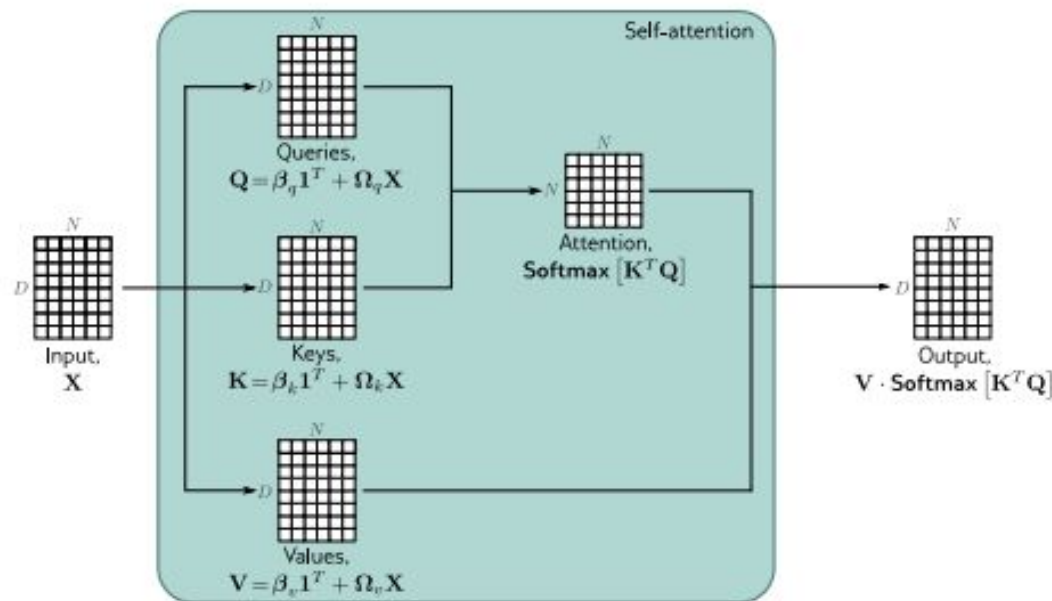
$$\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$$

$$\mathbf{k}_m = \beta_k + \Omega_k \mathbf{x}_m$$

$$\begin{aligned} a[\mathbf{x}_m, \mathbf{x}_n] &= \text{softmax}_m [\mathbf{k}_m^T \mathbf{q}_n] \\ &= \frac{\exp [\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^N \exp [\mathbf{k}_{m'}^T \mathbf{q}_n]} \end{aligned}$$

dot-product self-attention

Transformer: Dot Product Self Attention (3)



$$\mathbf{V}[\mathbf{X}] = \beta_v \mathbf{1}^T + \Omega_v \mathbf{X}$$

$$\mathbf{Q}[\mathbf{X}] = \beta_q \mathbf{1}^T + \Omega_q \mathbf{X}$$

$$\mathbf{K}[\mathbf{X}] = \beta_k \mathbf{1}^T + \Omega_k \mathbf{X}$$

$$\text{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \text{Softmax}[\mathbf{K}[\mathbf{X}]^T \mathbf{Q}[\mathbf{X}]],$$

Scaled dot product self attention:

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}\left[\frac{\mathbf{K}^T \mathbf{Q}}{\sqrt{D_q}}\right]$$

Figure 12.4 Self-attention in matrix form. Self-attention can be implemented efficiently if we store the N input vectors $\mathbf{x}_n$ in the columns of the $D \times N$ matrix $\mathbf{X}$. The input $\mathbf{X}$ is operated on separately by the query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$. The dot products are then computed using matrix multiplication, and a softmax operation is applied independently to each column of the resulting matrix to calculate the attentions. Finally, the values are post-multiplied by the attentions to create an output of the same size as the input.

Attention pada RNN:

Bahdanau Attention

h: hidden state pada encoder; j: time step pada encoder
s: hidden state pada decoder; t: time step pada decoder

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{N_x} \exp(e_{tk})}$$

$$e_{tj} = \text{align}(s_{t-1}, h_j) \\ = v_a^T \tanh(W_a s_{t-1} + U_a h_j)$$

$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

Luong Attention

$$\alpha_{tj} = \frac{\exp(\text{score}(s_t, h_j))}{\sum_{k=1}^{N_x} \exp(\text{score}(s_t, h_k))}$$

$$\text{score}(s_t, h_j) = \begin{cases} s_t^T h_j & \text{dot} \\ s_t^T W_a h_j & \text{general} \\ v_a^T \tanh(W_a [s_t; h_j]) & \text{concat} \end{cases}$$

$$c_t = \sum_{j=1}^N \alpha_{tj} h_j$$

Transformer

Self attention (dr satu input yg sama)

$$\mathbf{v}_m = \beta_v + \Omega_v \mathbf{x}_m$$

$$\text{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N] = \sum_{m=1}^N a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m.$$

Attention diambil dari transformasi linear thd input:

$$\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$$

$$\mathbf{k}_m = \beta_k + \Omega_k \mathbf{x}_m,$$

$$a[\mathbf{x}_m, \mathbf{x}_n] = \text{softmax}_m [\mathbf{k}_m^T \mathbf{q}_n] \\ = \frac{\exp [\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^N \exp [\mathbf{k}_{m'}^T \mathbf{q}_n]}$$

Hitungan self attention final

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}].$$

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Vaswani, A. (2017). Attention is all you need. *Advances in neural information processing systems*, 30, 1.

... We propose a new simple network architecture, the Transformer, based solely on **attention** mechanisms, dispensing with **recurrence** and **convolutions** entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train ...

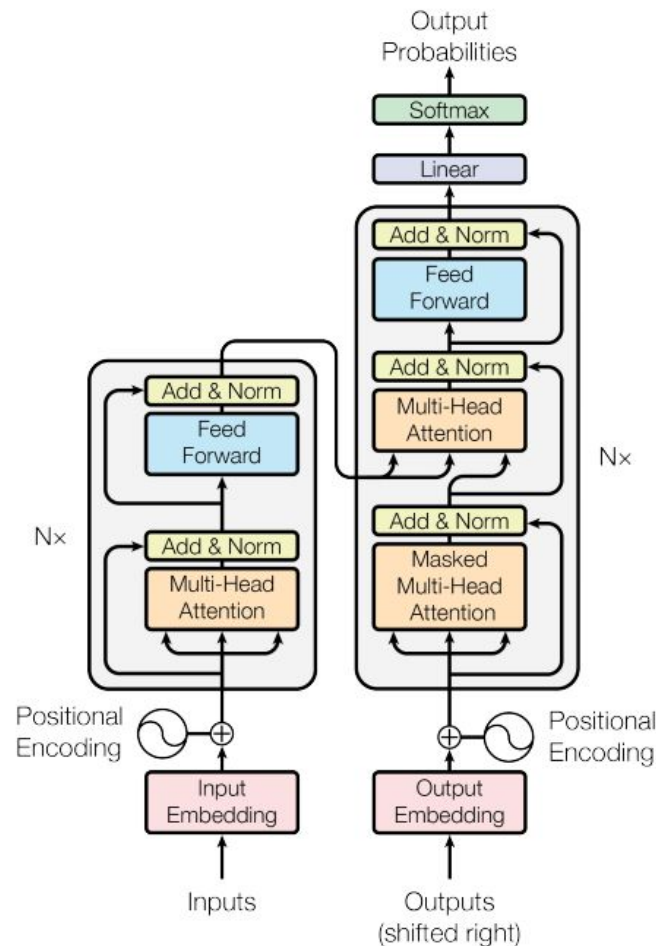


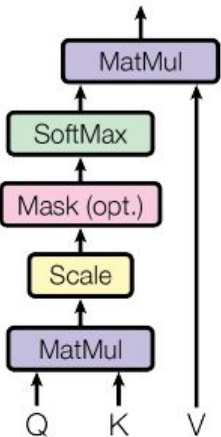
Figure 1: The Transformer - model architecture.

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

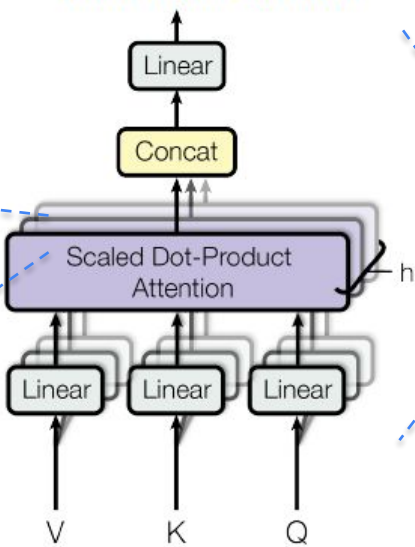
In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention

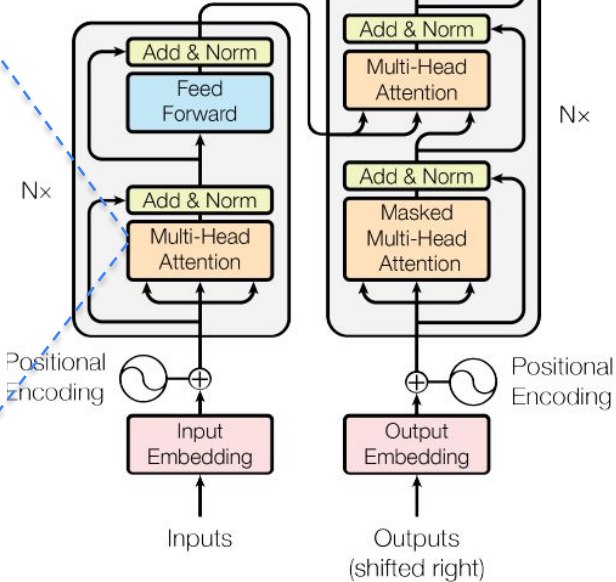


Multi-Head Attention



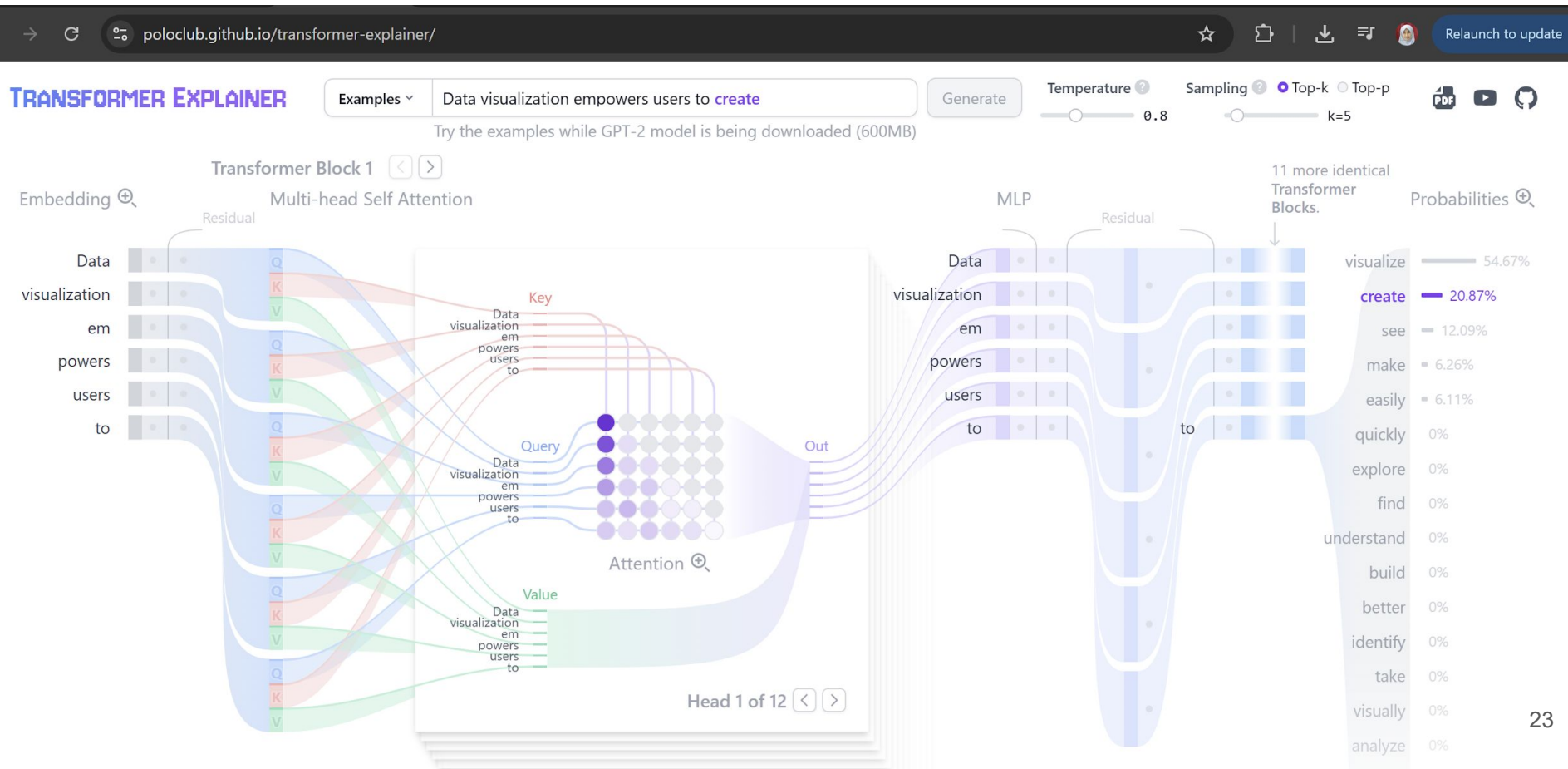
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



The Transformer - model architecture.

<https://poloclub.github.io/transformer-explainer/>



<https://jalammar.github.io/illustrated-transformer/>

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

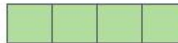
Softmax

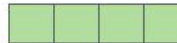
X
Value

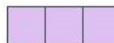
Sum

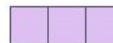
Thinking

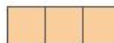
Machines

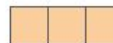
x_1 

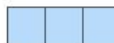
x_2 

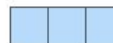
q_1 

q_2 

k_1 

k_2 

v_1 

v_2 

$q_1 \cdot k_1 = 112$

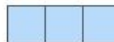
$q_1 \cdot k_2 = 96$

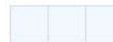
14

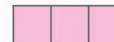
12

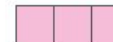
0.88

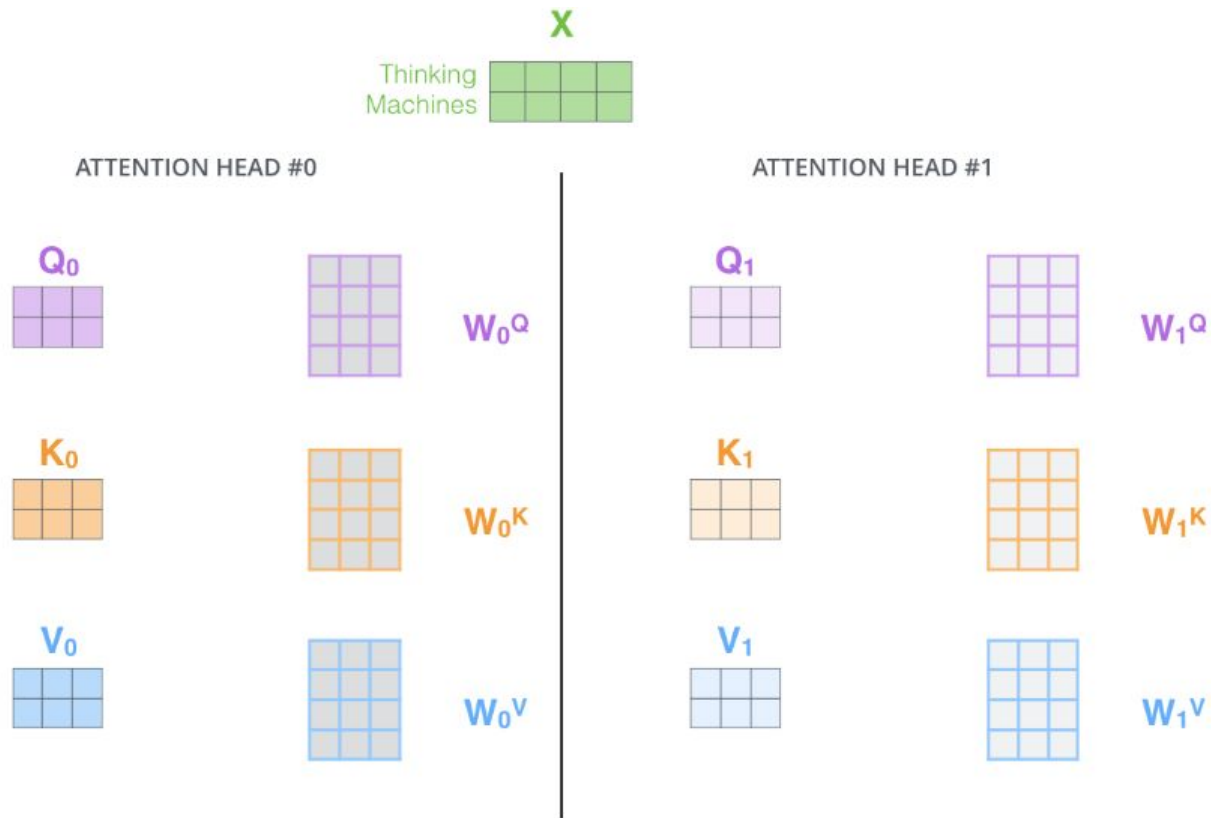
0.12

v_1 

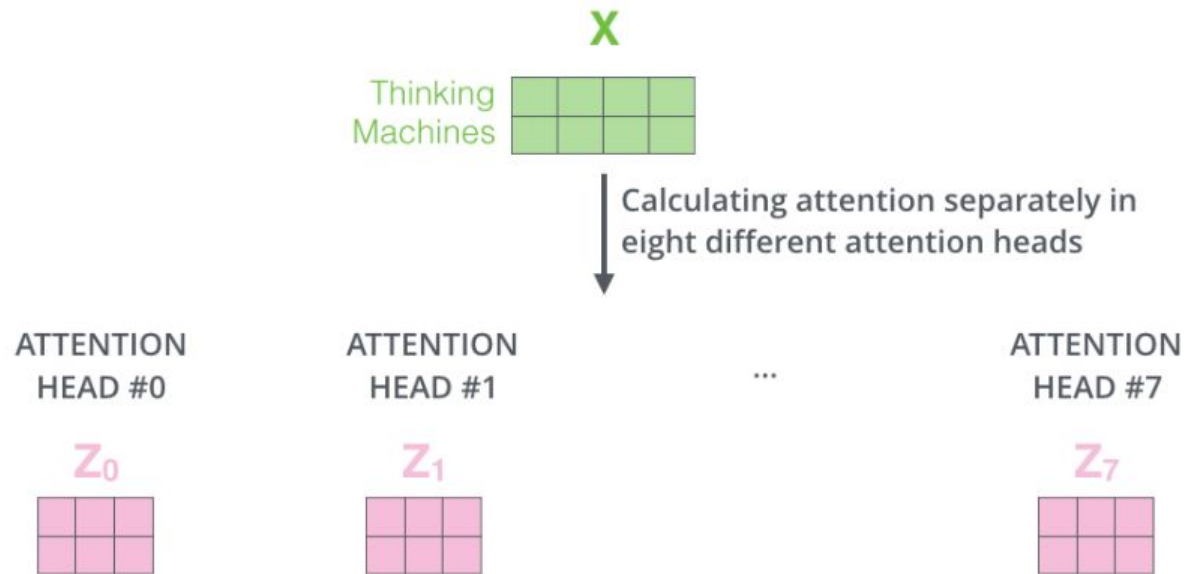
v_2 

z_1 

z_2 



With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the $W^Q/W^K/W^V$ matrices to produce Q/K/V matrices.



Z_0	Z_1	Z_2	Z_3	Z_4	Z_5	Z_6	Z_7

X

$$= \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array}$$

WO

1) This is our input sentence*

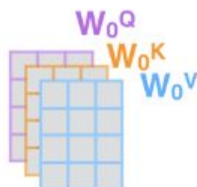
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

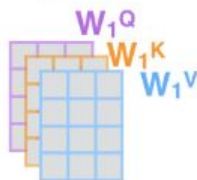
Thinking Machines



W^O



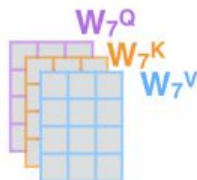
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



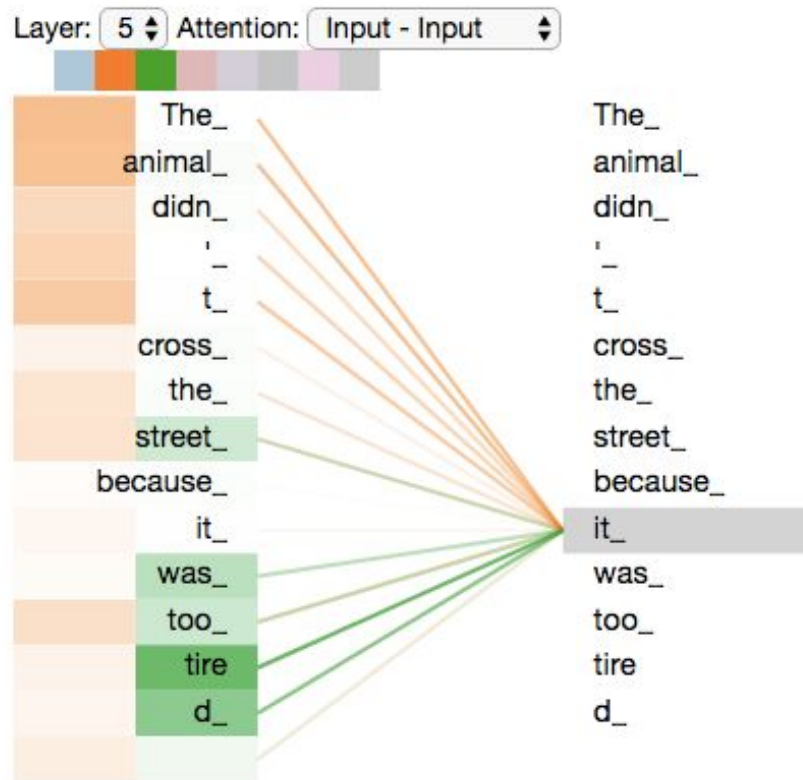
...

...

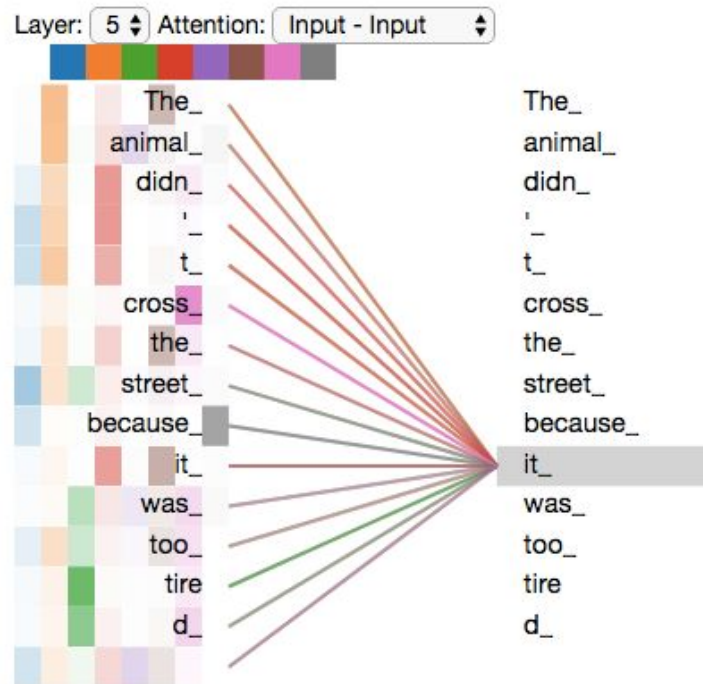
...



As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired"



If we add all the attention heads to the picture, however, things can be harder to interpret:



Positional Encoding

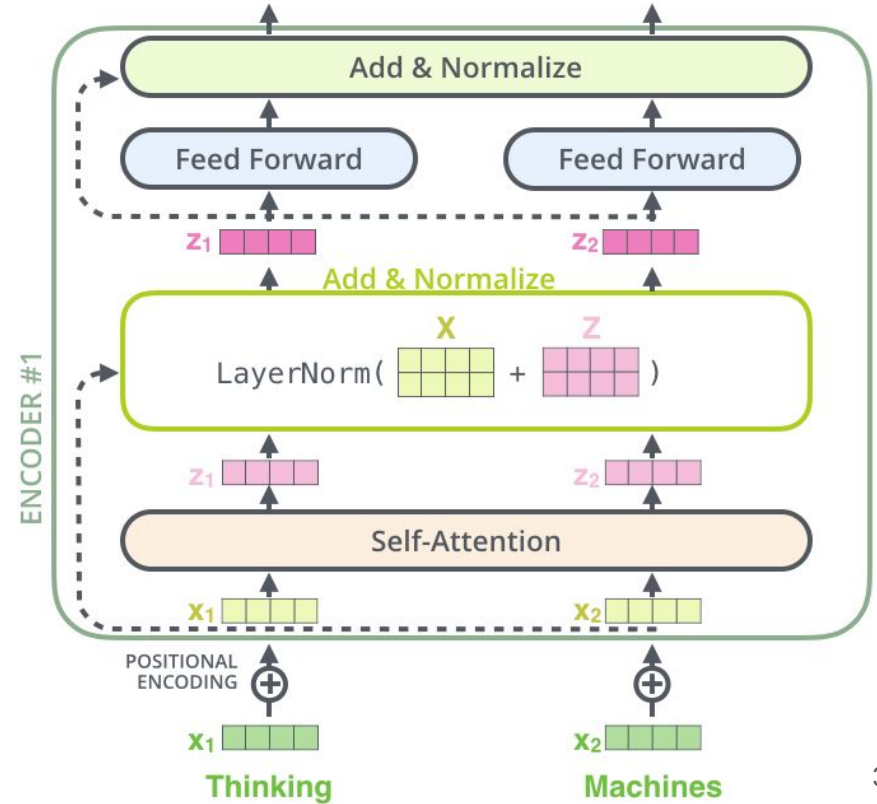
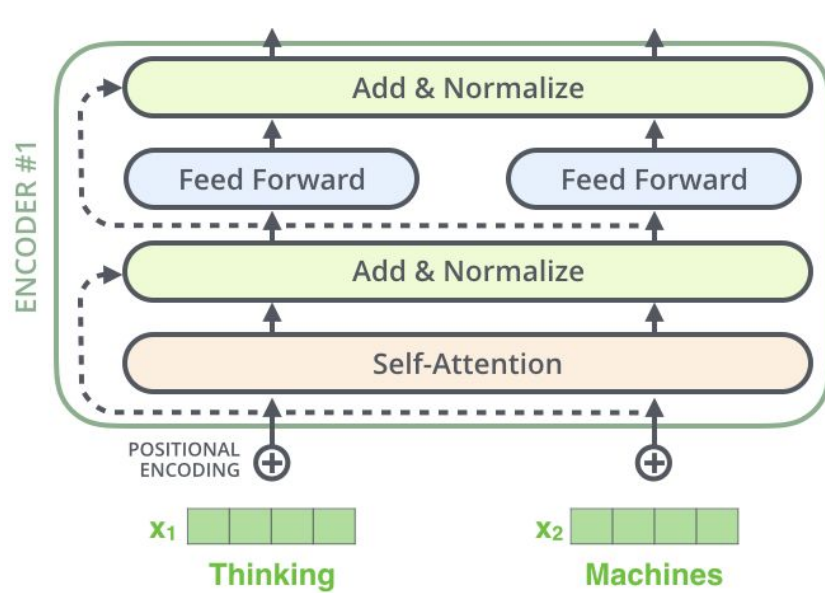
If we assumed the embedding has a dimensionality of 4, the actual positional encodings would look like this:



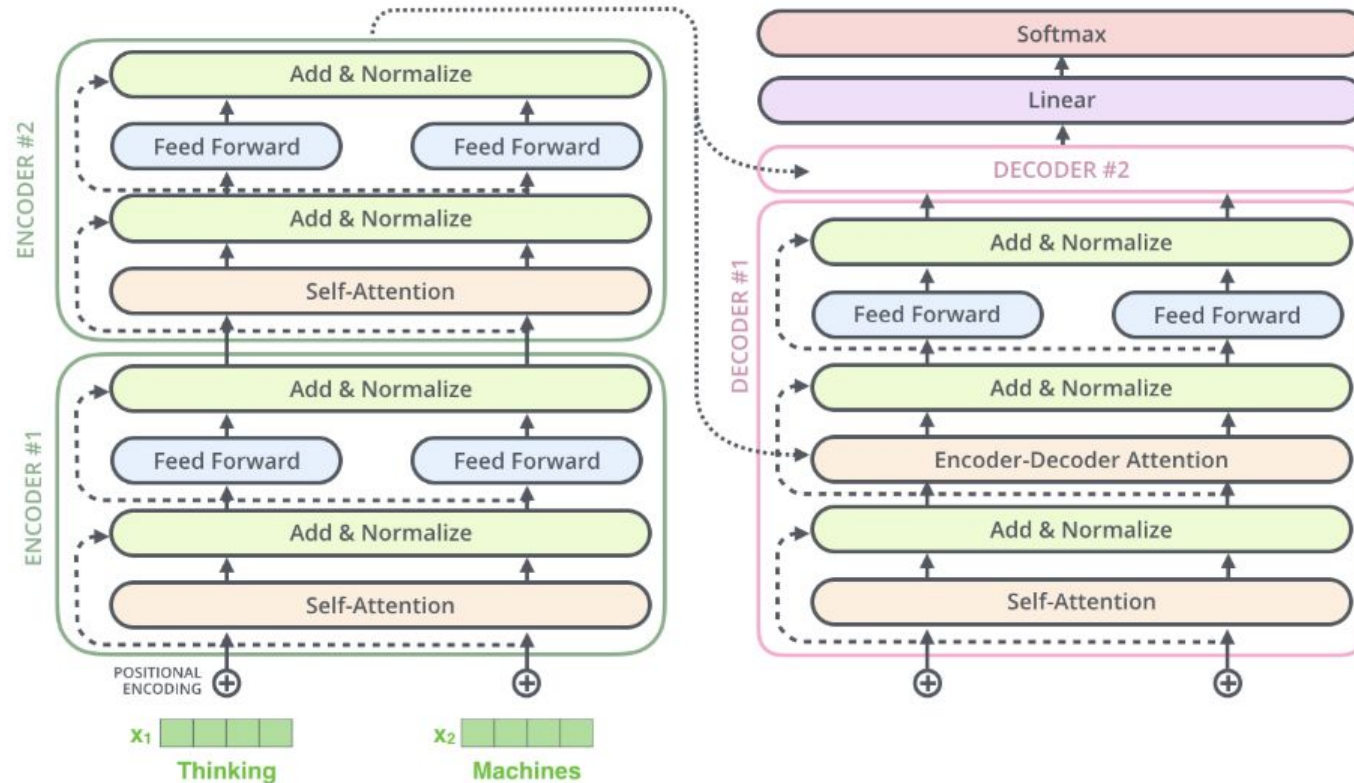
A real example of positional encoding with a toy embedding size of 4

The Residuals

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffn) in each encoder has a residual connection around it, and is followed by a [layer-normalization](#) step.

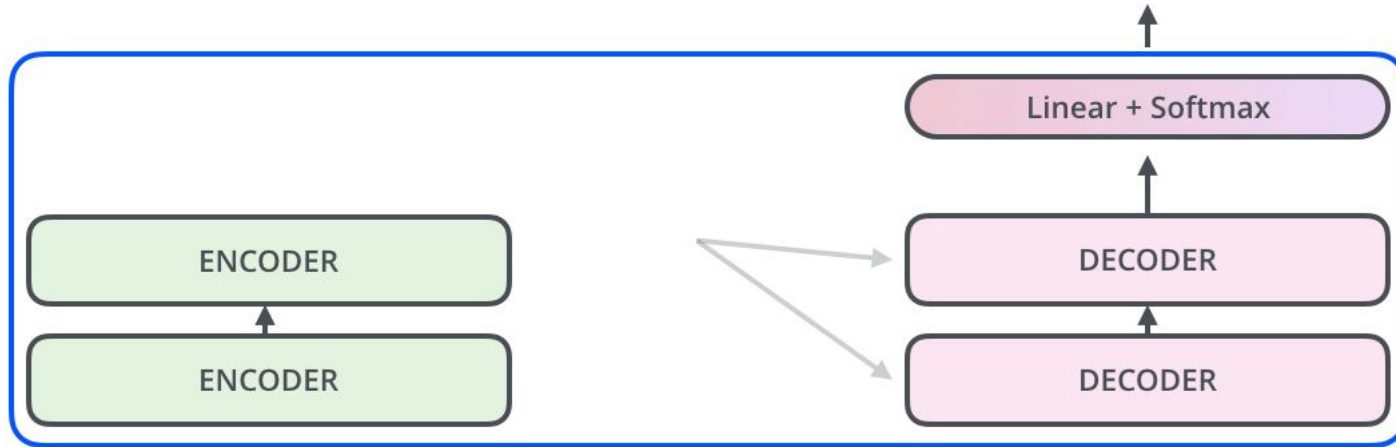


This goes for the sub-layers of the decoder as well. If we're to think of a Transformer of 2 stacked encoders and decoders, it would look something like this:



Decoding time step: 1 2 3 4 5 6

OUTPUT



EMBEDDING
WITH TIME
SIGNAL



EMBEDDINGS



INPUT

Je suis étudiant

The Linear layer is a simple fully connected neural network that projects the vector produced by the stack of decoders, into a much, much larger vector called a logits vector.

Let's assume that our model knows 10,000 unique English words (our model's "output vocabulary") that it's learned from its training dataset. This would make the logits vector 10,000 cells wide – each cell corresponding to the score of a unique word. That is how we interpret the output of the model followed by the Linear layer.

The softmax layer then turns those scores into probabilities (all positive, all add up to 1.0). The cell with the highest probability is chosen, and the word associated with it is produced as the output for this time step.

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(argmax)

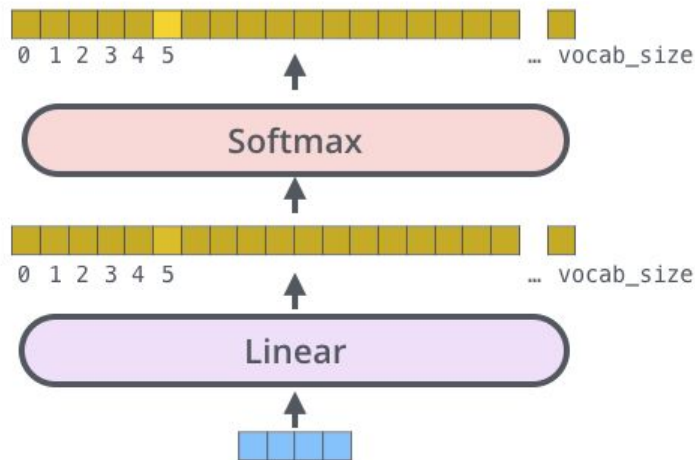
log_probs

logits

Decoder stack output

am

5



this figure starts from the bottom with the vector produced as the output of the decoder stack. It is then turned into an output word.